

AI Assignment #1

FasterNet for Robust Digit Pattern Recognition
under Geometric and Noise Corruptions

CHE CHI HIN, Angus
Student ID: UC325182
CISC3024 Pattern Recognition

Abstract

This report follows the required AI-assignment structure: (1) how AI was asked to find the algorithm, (2) algorithm description, (3) how AI implemented it, (4) experiment settings and results, (5) what was learnt, and (6) the source-code webpage link. The selected Deep CNN is **FasterNet** (CVPR 2023) with Partial Convolution (PConv), applied to robust digit recognition (0–9) on MNIST, EMNIST Digits and SVHN under rotation/noise/blur/shift/occlusion/elastic corruptions. All search, coding, experiments and drafting were performed by a Cursor AI agent under human direction (no hand-written code).

1 How I Asked AI Tools to Find the Algorithm

I used the **Cursor** AI coding agent as the only AI tool for search, programming and report writing. I did *not* hand-write code. Typical prompts / directions included:

- Find a **recent** Deep CNN or Deep Autoencoder (preferably 2023–2024) suitable for computer vision / pattern recognition; prefer peer-reviewed papers (CVPR/ICCV/NeurIPS).
- Avoid duplicating a peer’s **RepViT** + tiny 8×8 digit fusion narrative; choose a *different* mechanism and a standard digit benchmark story.
- Prefer an efficient CNN whose core operator can be verified (not only “train and report accuracy”).
- Frame the application as **robust digit pattern recognition** (rotated/noisy digits still recognized), not only clean upright MNIST.

The agent web-searched recent lightweight CNNs and proposed candidates. We locked **FasterNet** (Chen et al., CVPR 2023, arXiv:2303.03667) because Partial Convolution (PConv) is a clear, recent Deep-CNN contribution distinct from structural re-parameterization (e.g., MobileOne/RepViT). Rejected alternatives included simply copying RepViT. A command-level diary is kept in `process_log.md`.

2 Algorithm Description

FasterNet / PConv.. FasterNet argues that reducing FLOPs alone (e.g., depthwise convolution) may not maximize latency because of memory access cost [1]. **Partial Convolution** applies a spatial 3×3 convolution to only a fraction $r=1/n_{\text{div}}$ of channels (paper default $r=1/4$) and leaves the remaining channels untouched; a residual pointwise MLP then mixes channels.

Our adaptation (FasterNet-mini).. For 28×28 greyscale (MNIST/EMNIST) and 32×32 RGB (SVHN) we use a 2×2 patch stem and three stages (embed 32, depths (1, 2, 2)). Each block is $\text{PConv}(n_{\text{div}}) \rightarrow \text{BN} + \text{activation} + \text{pointwise MLP}$. Main runs use ReLU; we ablate GELU. We also implement a student-scale **p4 orbit-convolution** baseline for literature comparison (augmentation vs architectural invariance) [7].

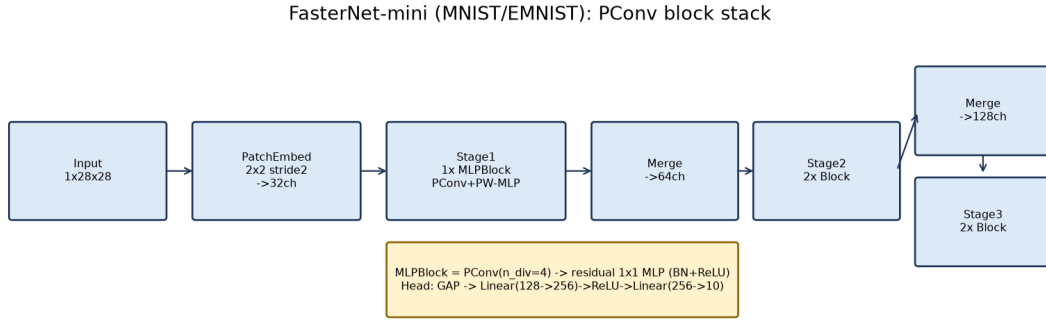


Figure 1: FasterNet-mini architecture used for MNIST/EMNIST.

PConv verification.. Untouched channels remain exact identities (max abs error = 0). At $n_{div}=4$, PConv uses 6.25% of dense 3×3 FLOPs on a fixed activation shape.

Related context.. MobileOne [2] targets mobile latency via re-parameterization (different mechanism). Corruption-style evaluation follows Hendrycks & Dietterich [3]. Digit domains: MNIST [4], EMNIST Digits [5], SVHN [6].

3 How AI Implemented the Algorithm

Under my direction, the Cursor agent implemented the full pipeline with **no hand-written student code**:

- `models/fasternet_mini.py` — PConv, MLPBlock, FasterNet-mini / T0-lite, GELU option.
- `models/equivariant_cnn.py` — p4 orbit-CNN baseline; `models/baseline_cnn.py` — SimpleCNN control.
- `corruptions.py` — train-time robust aug + test corruptions (rotate/noise/blur/shift/occlusion/elastic).
- `train.py` / `evaluate.py` — MNIST/EMNIST/SVHN loaders, MPS training, Rot90 TTA, reject curves, figures JSON.
- Analysis: `pconv_efficiency.py`, `analyze_extra.py`, `make_rich_figures.py`.
- Drivers: `run_all.py`, `run_upgrade_15.py`, `run_upgrade_gelu_eq_svhn.py`.
- Report/demo: `report.tex`, `app_gradio.py`, `process_log.md`.

I set goals, approved FasterNet, requested EMNIST/SVHN, multi-seed stats, GELU/equivariant/longer SVHN, and PDF polish; the agent wrote, ran and iterated the code.

4 Experiment Settings and Results

Settings. AdamW (10^{-3}), cosine schedule, batch 128, Apple MPS. Robust-aug includes cardinal/continuous rotations and noise/blur/shift/occlusion/elastic. EMNIST/SVHN use 60k/10k student-scale subsamples. Metrics: corruption accuracy, Rot90+TTA, reject@0.7, ECE, multi-seed mean $\pm$ std; ablations on n_{div} , GELU and equivariant baseline; SVHN 12 vs 24 epochs.

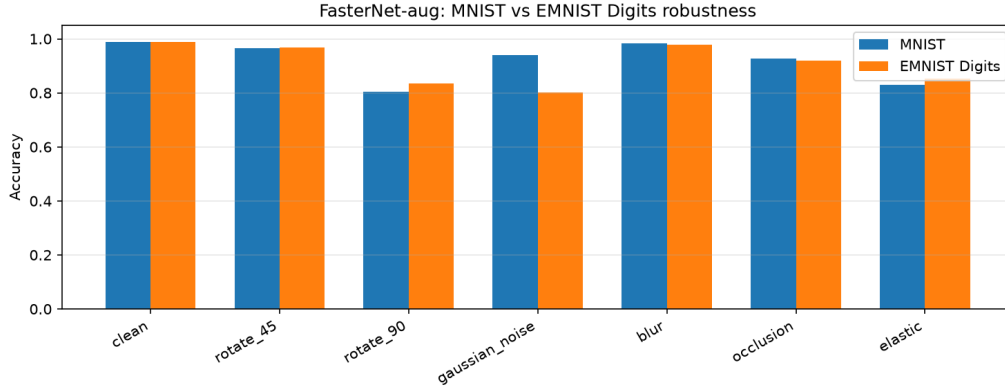
Table 1: Main robustness results.

Setting	Clean	Rot45	Rot90	Rot90+TTA	Noise
MNIST FasterNet-aug	0.989	0.965	0.805	0.939	0.940
EMNIST FasterNet-aug	0.988	0.968	0.835	0.941	0.801
MNIST clean-only	0.992	0.548	0.130	–	0.425
MNIST Baseline-aug	0.958	0.735	0.267	–	0.934
SVHN 12ep	0.789	0.576	0.572	0.720	0.641
SVHN 24ep long	0.858	0.711	0.671	0.792	0.693
GELU (MNIST)	0.988	0.973	0.866	0.935	0.857
p4 Equivariant-aug	0.964	0.912	0.823	0.927	0.831

Multi-seed MNIST Rot90: $84.1\% \pm 10.0\%$. EMNIST multi-seed Rot90: $81.2\% \pm 3.2\%$. Clean ECE ≈ 0.0076 ; hardest Rot90 pair $6 \leftrightarrow 9$. Full $n_{\text{div}} \in \{2, 4, 8\}$ 12-epoch ablation (Table 2) supports paper default $r=1/4$ for latency/clean trade-off.

Table 2: n_{div} ablation (12-epoch robust-aug MNIST).

n_{div}	Params	Clean	Rot45	Rot90	Noise	Lat. (ms/512)
2	342k	0.988	0.967	0.834	0.935	14.5
4	271k	0.987	0.966	0.887	0.885	12.1
8	253k	0.977	0.965	0.930	0.838	15.5

**Figure 2:** MNIST vs EMNIST Digits robustness.

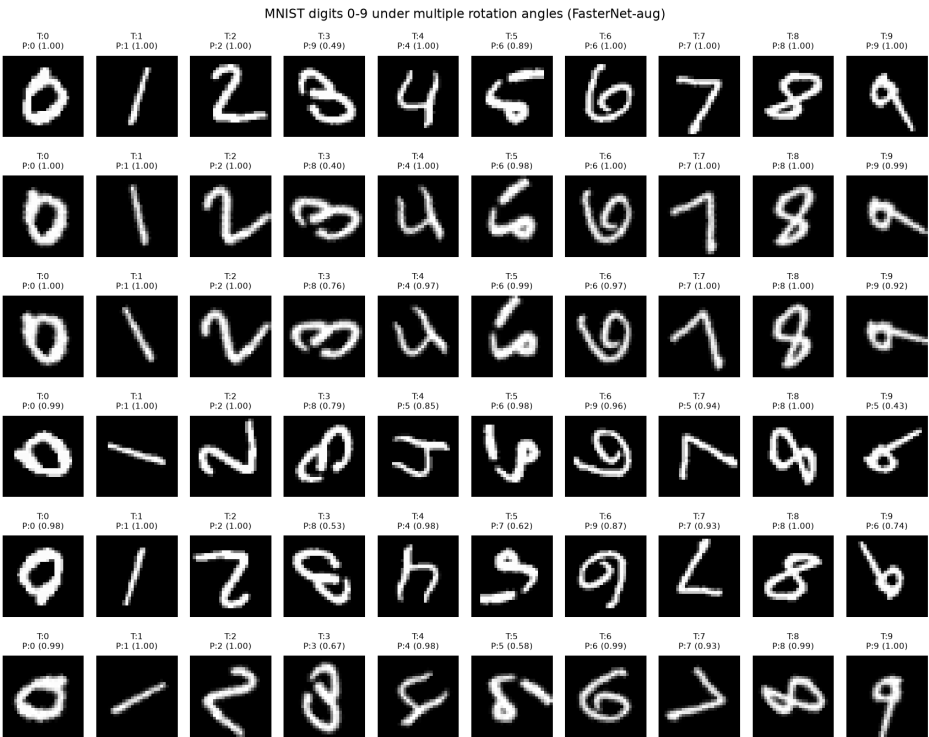


Figure 3: MNIST digits 0-9 under multiple rotation angles.

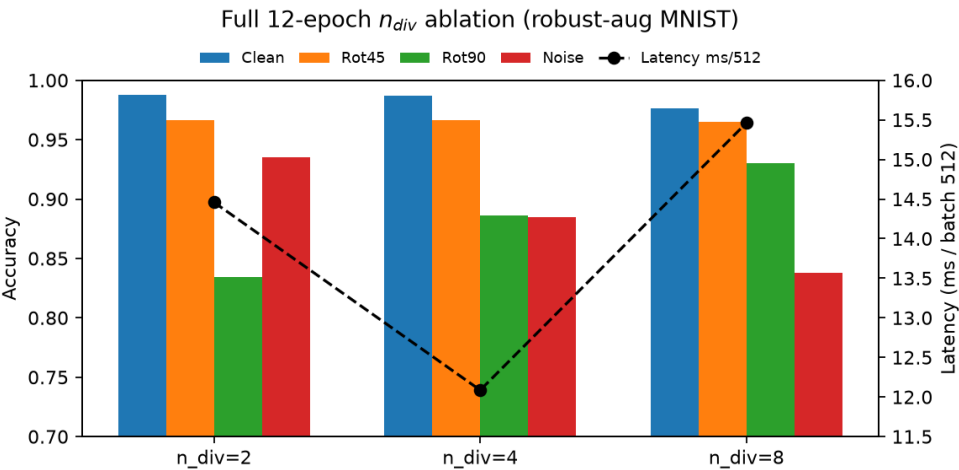


Figure 4: n_{div} accuracy-latency trade-off.

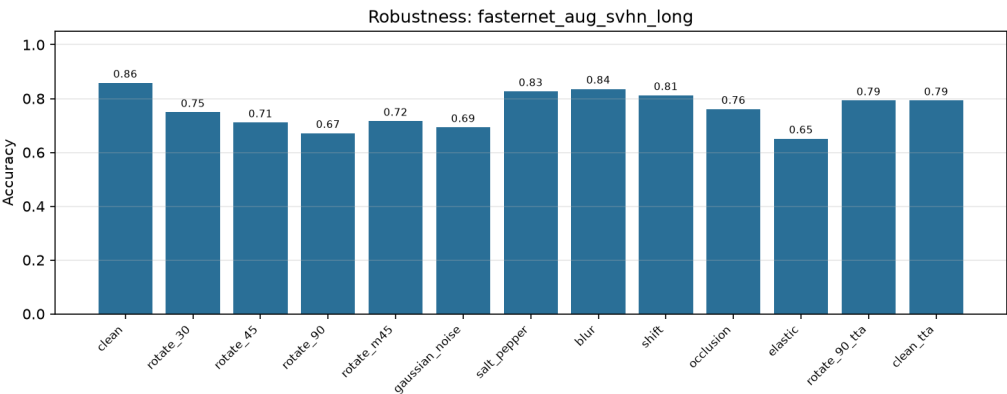


Figure 5: SVHN corruption bars after 24-epoch training.

5 What I Have Learnt from This AI Assignment

- **Directing AI is the skill:** clear constraints (recent Deep CNN, no peer-copy, robust digits, verify PConv) matter more than writing every line myself.
- **Clean accuracy is not enough** for pattern recognition under nuisance transforms; clean-only Rot90 collapses to $\sim 13\%$ while robust aug + TTA recovers to $\sim 80\text{--}94\%$.
- **Mechanism checks** (PConv identity/FLOPs, n_{div} ablation, multi-seed mean $\pm$ std) make an AI-generated project academically credible.
- **Literature alternatives** (equivariant CNNs) are useful baselines; for this budget, FasterNet + aug + TTA remained the better primary story.
- **Domain shift:** EMNIST writer variation and SVHN scene digits expose limits that upright MNIST hides; longer SVHN training (12 $\rightarrow$ 24 epochs) recovered large gains.
- **Process documentation** (`process_log.md`) is required and also helps debugging AI iterations.

6 Source Code Webpage Link

Source code for this assignment (AI-generated implementation, scripts, report sources):

<https://github.com/AngusJai/CISC3024-AI-Assignment1>

Repository includes `models/`, training/evaluation scripts, `process_log.md`, and `report.tex`. Large datasets under `data/` and local `.venv/` are excluded via `.gitignore`; download MNIST/EMNIST/SVHN with the provided scripts.

7 Conclusion

Using Cursor end-to-end, I searched for a recent Deep CNN (FasterNet), described and verified PConv, had AI implement the full robust-digit pipeline, reported experiments on MNIST/EMNIST/SVHN with ablations, reflected on learning outcomes, and published the source code link above.

References

- [1] J. Chen et al. Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks. CVPR, 2023.
- [2] P. K. A. Vasu et al. MobileOne: An Improved One Millisecond Mobile Backbone. CVPR, 2023.
- [3] D. Hendrycks and T. Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. ICLR, 2019.
- [4] Y. LeCun et al. Gradient-based learning applied to document recognition. Proc. IEEE, 1998.
- [5] G. Cohen et al. EMNIST: an extension of MNIST to handwritten letters. 2017.
- [6] Y. Netzer et al. Reading Digits in Natural Images with Unsupervised Feature Learning. NIPS Workshop, 2011.
- [7] M. Rath and A. Condurache. Deep Neural Networks with Efficient Guaranteed Invariances. AIS-TATS, 2023.
- [8] M. Rath and A. Condurache. Improving the Sample-Complexity of Deep Classification Networks with Invariant Integration. arXiv:2202.03967, 2022.